**NAME: S.JAYA SHREE** **REG NO: 192524146**

**EXPERIMENT 8:**

**Cyclomatic Complexity in Software Testing is a testing metric used for measuring the complexity of a software program. It is a quantitative**

**measure of independent paths in the source code of a software program.**

**Cyclomatic complexity can be calculated by using control flow graphs or**

**with respect to functions, modules, methods or classes within a software**

**program. Cyclomatic complexity for a flow graph G is V(G)=E-N+2.**

**Cyclomatic complexity is a software metric (measurement), used to**

**indicate the complexity of a program, Cyclomatic complexity = E - N +**

**P where, E = number of edges in the flow graph, N = number of nodes**

**in the flow graph, P = number of nodes that have exit points. Find**

**Cyclomatic Complexity for a graph having number of edges as 17,**

**number of nodes as 13 and number of predicate nodes in the flow graph**

**as 5.**

# Aim

The objective of this experiment is to calculate the **Cyclomatic Complexity** of a control flow graph using the given number of edges, nodes, and predicate nodes, and to understand the complexity of a software program.

# Software Requirements

**Software:** RAPTOR / Flowchart Tool (Optional)
**Operating System:** Windows 10/11
**Hardware:** PC/Laptop
**Calculator:** For manual computation

# Theory

Cyclomatic Complexity is a software testing metric used to measure the complexity of a program. It represents the number of independent execution paths in a program and helps determine the minimum number of test cases required for complete path coverage. It is calculated using the Control Flow Graph (CFG), where nodes represent program statements and edges represent the flow of control.

The Cyclomatic Complexity can be calculated using either of the following formulas:

- **V(G) = E − N + 2P**
- **V(G) = Predicate Nodes + 1**

where:

- **E** = Number of edges
- **N** = Number of nodes
- **P** = Number of connected components (usually 1 for a single program)

A higher Cyclomatic Complexity indicates a more complex program that requires additional testing and maintenance.

# Procedure

**Step 1:**

Identify the given values from the control flow graph.

- Number of Edges (E) = **17**
- Number of Nodes (N) = **13**
- Number of Predicate Nodes = **5**

**Step 2:**

Apply the Cyclomatic Complexity formula:

**V(G) = E − N + 2P**

Since the graph has one connected component,

**P = 1**

**Step 3:**

Substitute the given values:

**V(G) = 17 − 13 + 2(1)**

**Step 4:**

Calculate the result:

**V(G) = 17 − 13 + 2 = 6**

**Step 5:**

Verify the result using the alternate formula:

**V(G) = Predicate Nodes + 1**

**V(G) = 5 + 1 = 6**

Both methods produce the same Cyclomatic Complexity value.

# Observation

| Parameter | Value |
|---|---|
| Number of Edges (E) - | 17 |
| Number of Nodes (N) - | 13 |
| Predicate Nodes - | 5 |
| Connected Components (P) | - 1 |
| Cyclomatic Complexity - | 6 |

# Calculation

$$V(G) = E - N + 2P$$
$$V(G) = 17 - 13 + 2(1)$$
$$V(G) = 6$$

Also,

$$V(G) = \text{Predicate Nodes} + 1 = 5 + 1 = 6$$

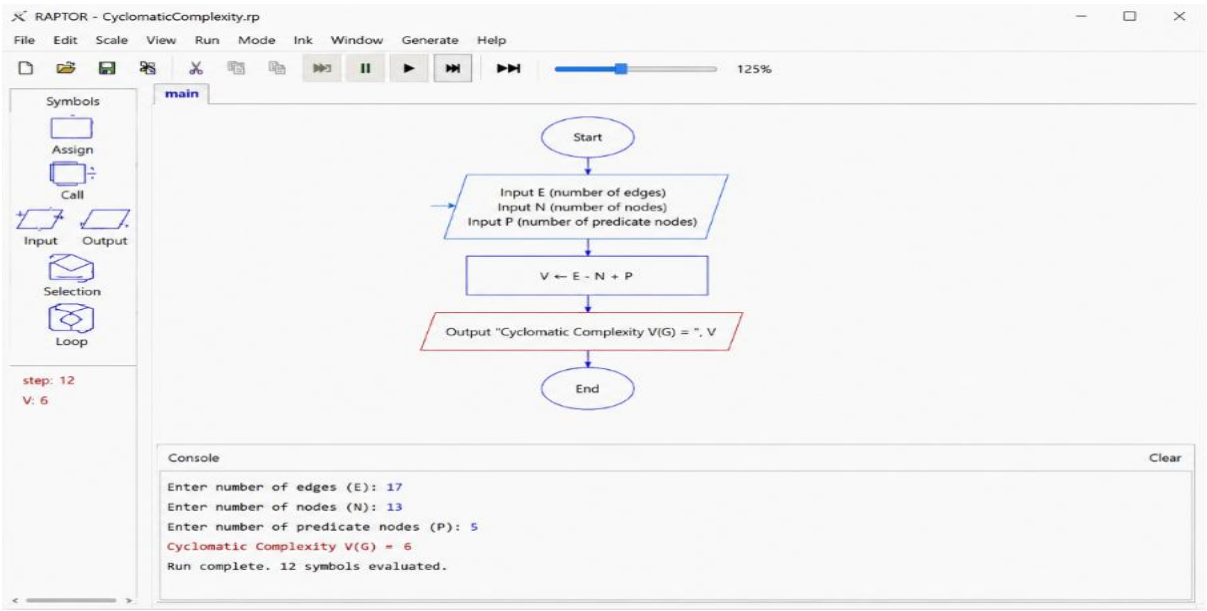


*Figure 1. Execution of the Flowchart.*

# Result

The Cyclomatic Complexity of the given control flow graph was calculated successfully. The complexity value is **6**, indicating that there are **6 independent execution paths** in the program. Therefore, a minimum of **6 test cases** is required to achieve complete basis path testing.